

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФГБОУ ВО Волгоградский государственный технический университет
Кафедра «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ АВТОМАТИЗИРОВАННЫХ
СИСТЕМ»

Управление ветвлением в распределенных системах контроля версий

Методические указания



Волгоград
2018

Рецензент

к. т. н., доц. Н.П. Садовникова

Издается по решению редакционно-издательского совета
Волгоградского государственного технического университета

Управление ветвлением в распределенных системах контроля версий: метод. указания / сост. О. А. Сычев, Г.В. Терехов; ВолгГТУ. – Волгоград, 2018. – 16 с.

В методических указаниях изложены цели, содержание и порядок выполнения лабораторных работ «Создание и слияние веток в проекте с общим центральным репозиторием» и «Создание и слияние веток в проекте с индивидуальными репозиториями разработчиков» дисциплин «Эволюция и сопровождение программного обеспечения» и «Распределенный контроль версий кода», приведены основные сведения об управлении ветками и метками в современных системах распределенного контроля версий Git и Mercurial.

Методические указания предназначены для изучения дисциплин «Эволюция и сопровождение программного обеспечения» и «Распределенный контроль версий кода». Методические указания могут быть использованы студентами, обучающимися по направлению 09.03.04 «Программная инженерия», а также широким кругом лиц, интересующихся программированием.

© Волгоградский государственный
технический университет, 2018

Оглавление

1. Управление метками и ветками в Mercurial и Git.....	4
1.1 Стратегии и способы организации ветвления.....	4
1.2 Команды ветвления и слияния веток в Git.....	8
1.3 Команды ветвления и слияния веток в Mercurial.....	10
1.4 "Легкие" ветки в Mercurial.....	13
1.5 Работа с метками в Mercurial и Git.....	14

1. Управление метками и ветками в Mercurial и Git

В распределенных системах контроля версий команды ветвления и слияния веток существенно доработаны по сравнению с централизованными. Вызвано это тем, что простейший их вариант – анонимные ветки – появляются рутинным образом при вытягивании изменений из другого репозитория, если работа в этих двух репозиториях велась параллельно. При этом для успешного выталкивания результата на сервер эти ветки необходимо слить между собой.

Одной из интересных особенностей веток является контроль над слиянием созданных веток с выпускающей (релиз) веткой. Такое слияние является по сути окончательным разрешением для кода для попадания в релиз, поэтому сотрудник, который его выполняет, получает эффективно значительную долю властных полномочий. Начальник может, конечно, отменять неправильные слияния и коммиты постфактум – но на практике это занимает столько времени, что реализуется крайне редко – гораздо проще не выполнять слияние до достижения требуемого качества кода разработки чем отменять сделанные. Поэтому слияние в выпускающую ветку обычно выполняется руководителем (лидером) проекта или опытным сотрудником, ответственным за его качество.

1.1 Стратегии и способы организации ветвления

Ответ на вопросы «когда создавать и сливать ветки?» - стратегия ветвления – имеет большое значение для работы над кодом проекта. Правильный выбор стратегии ветвления с учетом характеристик сотрудников и работы над проектом способен сильно облегчить работу лидера проекта и улучшить качество кода, ошибочный же выбор вполне способен как добавить работы, так и отрицательно сказаться на результатах разработки и создать немало конфликтных ситуаций. Перечисленные ниже стратегии не исключают друг друга и могут использоваться в любых сочетаниях.

1) Ветки (клон) для каждого *разработчика* (developer-branch). В этом случае каждый разработчик производит коммиты только в свою ветку (чаще всего – отдельный серверный клон), куда они имеют право выталкивать; релиз-версия собирается руководителем или специально назначенным сотрудником в специальном клоне. Централизованный подход (когда все разработчики выталкивают свой код в одну ветку единого серверного репозитория) в настоящее время встречается все реже и реже (в основном в малых коллективах на небольших проектах), так как он приводит к значительному хаосу в репозитории и большому количеству мелких разветвлений и слияний. Кроме того, выталкивание каждым разработчиком своих изменений напрямую в ветку и репозиторий, в

которых подготавливается релиз-версия затрудняет выявление и исправление ошибок. Поэтому обычно для каждого разработчика выделяется отдельный серверный клон (потому, что на клоны можно настраивать права на чтение и запись, в то время как при выделении ветки для разработчика следование правилам остается на его совести), а руководитель или специально выделенный сотрудник вытягивают изменения из клонов разработчиков в основной (релиз-клон) и сливают их вместе. На рисунке 1 показан пример репозитория, в котором коммиты разработчика по имени John периодически сливаются из одноименной ветки в ветку master, в то время как коммиты разработчика Bob еще не готовы попасть в master, но его ветка своевременно актуализируется, затыкая коммиты из master. Код третьего разработчика, Alice, был слит в master, но после слияния эта ветка новых коммитов пока не содержит.

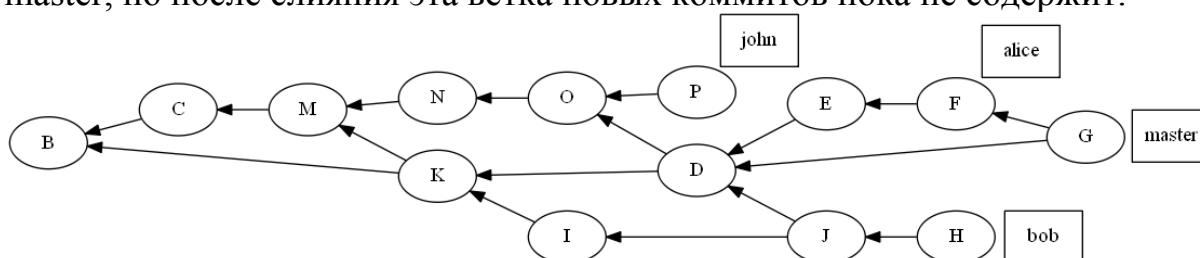


Рисунок 1. Пример реализации стратегии developer-branch

Ветки для каждой *большой версии* (major release-branch). Большими версиями (major release) обычно называют версии, приносящие значительные изменения в функциях (и в некоторых случаях API) программы, в отличие от малых (minor release), сконцентрированных только на исправлениях найденных ошибок и мелких совершенствованиях. При типовой ситуации в разработке ПО команде необходимо исправлять ошибки в одной-двух выпущенных ранее версиях (такие версии называются стабильными) и параллельно вести разработку новой большой версии (нестабильной). В этом случае логичным является создание ветки при каждом выпуске большого релиза, с тем, чтобы всегда можно было переключиться на эти ветки для исправления ошибок, обнаруженных пользователем. Малые релизы накапливают исправления ошибок в рамках одной стабильной версии и ветвления не требуют (см. рис. 2).

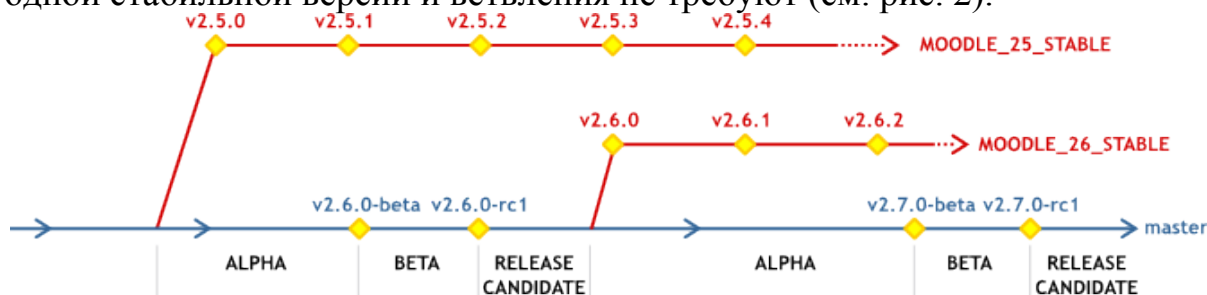


Рисунок 2. Пример реализации стратегии major release-branch

Рисунок 3. Пример реализации стратегии feature-branch

6

критически важную ошибку). Выделение отдельных веток для каждой задачи позволяет руководителю выборочно сливать только хорошо выполненные задачи, оставляя остальные не слитыми до полного исправления. На рисунке 4 показан пример репозитория, в котором задачи с трекера под номерами issue-51 и issue-56 решены в одноименных ветках и слиты с веткой master. Задача issue-53 требует дальнейшей доработки и еще не готова к слиянию в master, однако своевременно актуализируется, получая коммиты из ветки master.

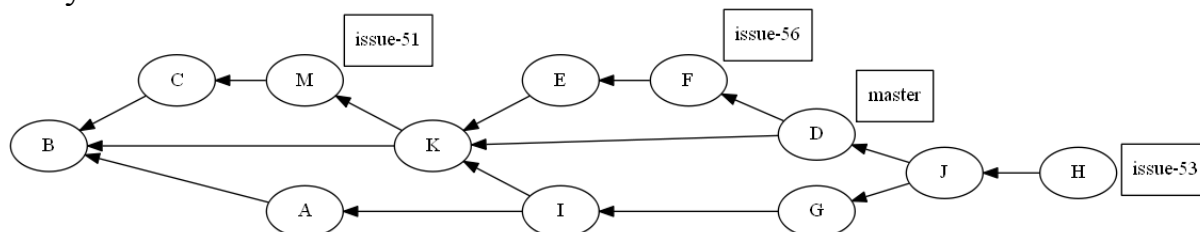


Рисунок 4. Пример реализации стратегии issue-branch

Всего существует 4 способа организации ветвления в распределенных системах.

- 1) Клонирование репозитория – любая копия репозитория может использоваться как ветка, в которой идет самостоятельная разработка. Основным недостатком данного метода является хранение копии совместной истории веток в каждом клоне, а также необходимость ручного выполнения команд для вытягивания (выталкивания в) каждого отдельного клона. Чаще всего этот подход применяется на уровне developer-branch, поскольку отдельный репозиторий-клон позволяет отдельно настроить права доступа для каждого разработчика, заставляя их соблюдать правила – кто и куда может выталкивать. Кроме того, существование отдельных репозиториях уменьшает количество разветвлений и слияний при параллельной работе нескольких программистов в рамках одной ветки.

- 2) «Тяжелые» ветки Mercurial – являются частью истории репозитория, их имя навсегда связывается историей кода, по крайней мере, на ее отрезке от разделения до слияния вместе.

- 3) Закладки в Mercurial, «легкие» ветки в Git – версионироваться как отдельные объекты и бесследно исчезают после удаления. В целом обычно производительнее легких веток.

- 4) Анонимные ветки – образуются естественным образом при вытягивании параллельно сделанных

изменений или коммите после обновления к набору изменений, не являющемуся головным. Хотя их возможности почти не отличаются от именованных веток, практическое применение для задач ветвления требует помнить хештэг (шестнадцатиричное уникальное число) последнего коммита в каждой ветке, что крайне неудобно. Поэтому, как правило, анонимное ветвление используется только для слияния своих изменений с изменениями других разработчиков, сделанными параллельно.

1.2 Команды ветвления и слияния веток в Git

В репозитории git всегда существует главная ветка (под названием master и default) соответственно. Для просмотра списка веток и уточнения текущей ветки в Git используется команда branch без параметров, которая выводит список веток и помечает активную ветку звездочкой (см. рис. 5).

```
$ git branch
* dev
Master
```

Рисунок 5. Пример использования команды branch

Если команде branch передать в качестве параметра имя ветки, то она будет создана. Для переключения между ветками используется команда checkout, ей также передается имя целевой ветки (см. рис. 6).

```
$ git branch foo
$ git checkout foo
Переключено на ветку «foo»
$ git branch
dev
* foo
Master
```

Рисунок 6. Пример переключения между ветками в Git

Для слияния веток используется команда merge. Для этого вы должны находиться в одной ветке и передать имя другой в качестве параметра команды merge. Например, если мы хотим слить ветку foo в главную master то нужно выполнить команды, представленные на рисунке 7.

```
$ git checkout master
Переключено на ветку «master»
$ git merge foo
```

Рисунок 7. Пример слияния веток в Git

Git автоматически создает коммит если слияние не вызывает конфликтов. В случае наличия конфликтов вы должны их разрешить, используя команду mergetool и произвести коммит вручную.

Обратите внимание, что слияние ветки не приводит к ее удалению – и в дальнейшем пути слитых веток снова могут разойтись. Удалить ставшую ненужной ветку можно с помощью команды `git branch` с указанием специальной опции и имени ветки. Для этого используются опция `-d` (удаляет ветку только если она была слита с другой веткой) или `-D` (удаляет ветку в любом случае, при этом код в ней будет потерян если ветка не была слита). Стратегии `feature-branch` и особенно `issue-branch` приводят к частой необходимости окончательного (с удалением) слияния веток, чтобы не загромождать репозиторий большим количеством уже ненужных имен веток. В этом случае использование опции `-d` представляет собой полезную предосторожность во избежание случайного удаления не слитой ветки с полезным кодом при рутинной чистке репозитория от старых веток.

Рассмотрим в качестве примера репозиторий, ветки в котором показаны на рисунке 8.

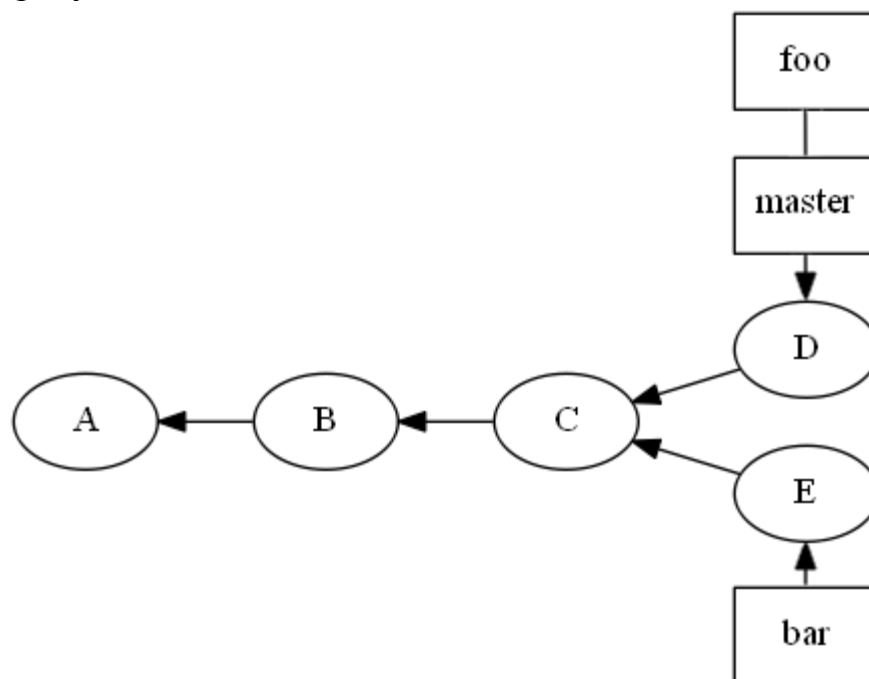


Рисунок 8. Пример веток репозитория

Как видно из рисунка, ветка «foo» указывает туда же, куда и «master» (например, после слияния). Если эта ветка нам не нужна, ее можно удалить, выполнив команду `git branch foo -d` (см. рис. 9).

```
$ git branch -d foo
Deleted branch foo (3a0874c).
```

Рисунок 9. Удаление ветки из репозитория при помощи команды `git branch foo -d`

Удалить ветку `bar` командой с опцией `-d` не получится (см. рис. 10).

```
$ git branch -d foo
error: The branch 'foo' is not fully merged.
If you are sure you want to delete it, run 'git branch -D foo'.
```

Рисунок 10. Попытка удалить не слитую с master ветку из репозитория при помощи команды `git branch foo -d`

Если мы хотим удалить ветку «bar» (что, в конце концов, приведет к удалению коммита E), необходимо выполнить команду `git branch bar -D` (см. рис. 11).

```
$ git branch -D bar
Deleted branch bar (3a0874c).
```

Рисунок 11. Удаление ветки из репозитория при помощи команды `git branch foo -D`

Git поддерживает отдельное понятие «удаленных» (remote) веток, которые отслеживают состояние некоторой ветки на удаленном сервере. Называться такая ветка будет по принципу «имя сервера/имя ветки» - например `origin/master` (`origin` – по умолчанию имя репозитория, из которого вы сделали клон своего). Команда `fetch` при получении изменений от удаленного репозитория обновляет именно такие ветки, оставляя ваши локальные копии этих веток нетронутыми. Команда `pull` позволяет получить изменения и автоматически слить их с вашими локальными ветками, отслеживающими (tracking branches) изменения в ветках на сервере. Обратите внимание, что в Git команды типа `push` и `pull` обычно используются с одной веткой. Если ваш репозиторий ранее не получал (используя команду `checkout`) какую-либо ветку с сервера, то команды `fetch` и `pull` не создадут ее автоматически у вас.

1.3 Команды ветвления и слияния веток в Mercurial

В репозитории Mercurial всегда существует главная ветка под названием `default`. Для просмотра списка веток и определения текущей ветки в Mercurial можно использовать команду `branches` без параметров (см. рис. 12).

```
$ hg branches
default                  153:835591ff7167
lualatex_testing         124:2ccaa4dbfac2 (неактивна)
$ hg branch
default
```

Рисунок 12. Просмотр веток в Mercurial

Если команде `branch` передать в качестве параметра имя ветки, то она будет создана. Для переключения между ветками используется команда `update`, при этом ей вместо ревизии передается имя целевой ветки (см. рис. 13).

```
$ hg branch foo
marked working directory as branch foo
$ hg branch
foo
```

```
$ hg checkout lualatex_testing
13 файлов обновлено, 0 слито, 12 удалено, 0 с конфликтами
$ hg branch
lualatex_testing
```

Рисунок 13. Пример создания ветки в Mercurial

Для слияния веток используется команда `merge`. Для этого вы должны находиться в одной ветке и передать имя другой в качестве параметра команды `merge`. Например, если мы хотим слить ветку `foo` в главную `master` то нужно выполнить команды, представленные на рисунке 14.

```
$ hg update default
17 файлов обновлено, 0 слито, 9 удалено, 0 с конфликтами
$ hg merge foo
$ 1 файлов обновлено, 0 слито, 0 удалено, 0 с конфликтами
$ hg commit -m 'Merge'
```

Рисунок 14. Пример слияние веток в Mercurial

Обратите внимание, что Mercurial не создает автоматически коммит слияния даже при отсутствии конфликтов, поэтому вы должны запустить команду коммита сами. Это может быть полезно, поскольку формальное отсутствие конфликтов слияния еще не означает, что слитый код будет корректно компилироваться и работать¹. Вы можете захотеть запустить юнит-тесты и произвести другие виды тестирования (или, по крайней мере, проверить что результат компилируется без ошибок) при слиянии (особенно в стратегиях с долгоживущими ветками типа `feature-branch`) и коммитить результат после исправления проблем, если они возникли.

Mercurial считает именованные ветки постоянными и не удаляемыми полностью из истории проекта, данные о ветке прописываются в наборе данных коммита и к ней всегда можно вернуться используя команду `update`. Поэтому именованные ветки рекомендуется создавать для долгосрочных проектов (стратегии `major release-branch`, `developer-branch`, иногда `feature-branch`). Поэтому удалить ветку полностью из репозитория нельзя, но можно ее закрыть, используя команду `commit` с параметром `--close-branch` (см. рис. 15).

```
$ hg commit --close-branch -m 'this branch no longer required'
```

Рисунок 15. Пример закрытия ветки в Mercurial

Если вы хотите полностью удалить ветку из репозитория, вы можете воспользоваться командой `clone` с опцией `--rev` так, чтобы не затягивать в новый репозиторий коммиты соответствующей ветки. Также

¹ Простейший пример такой ситуации – это когда в одной ветке изменяется интерфейс функции (вместе со всеми ее вызовами для сохранения работоспособности), а другая ветка вводит новые вызовы этой функции со старым интерфейсом. Обе ветки будут работать и сливаться без конфликтов, однако результат их слияния выдаст ошибку компиляции (или времени выполнения в интерпретируемом языке).

для команды `clone` можно использовать опцию `--branch`, явно указав ветки, которые нужно клонировать. Рассмотрим репозиторий, состояние которого отображено на рисунке 16.

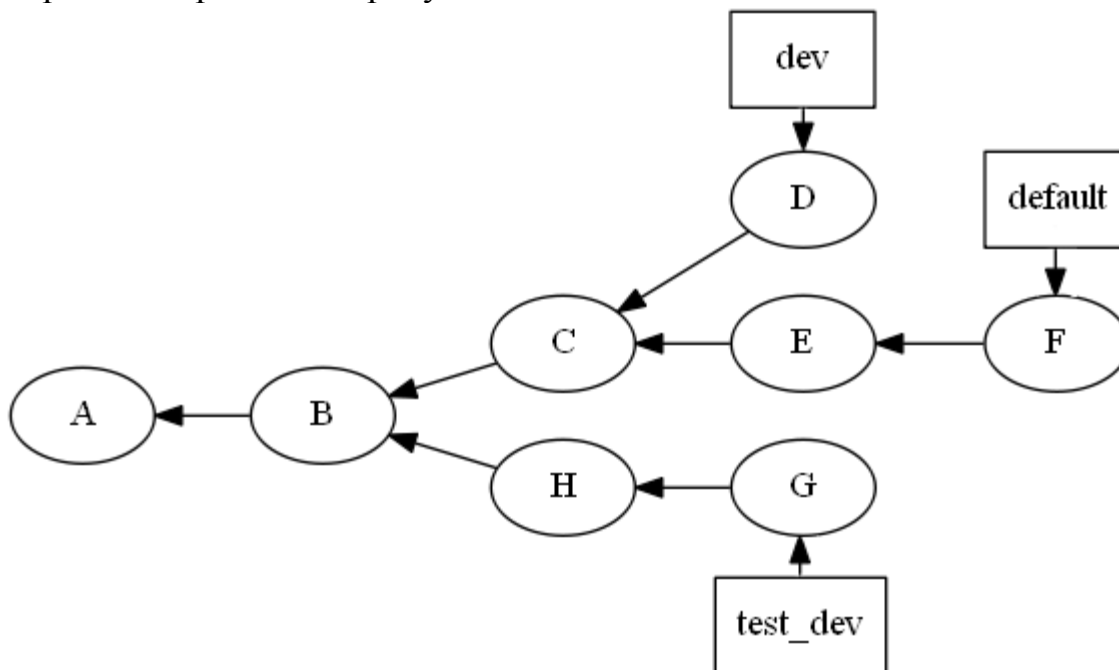


Рисунок 16. Пример состояния репозитория с тремя не слитыми ветками.

Нам нужны только ветки `default` и `dev`, ветка `test_dev` является для нас лишней. Клонировем репозиторий без нее, как показано на рисунке 17.

```
$ hg clone backup repo --branch default dev
```

Рисунок 17. Клонирование репозитория в Mercurial без ветки `dev_test`

После такого клонирования наш репозиторий будет выглядеть так, как показано на рисунке 18.

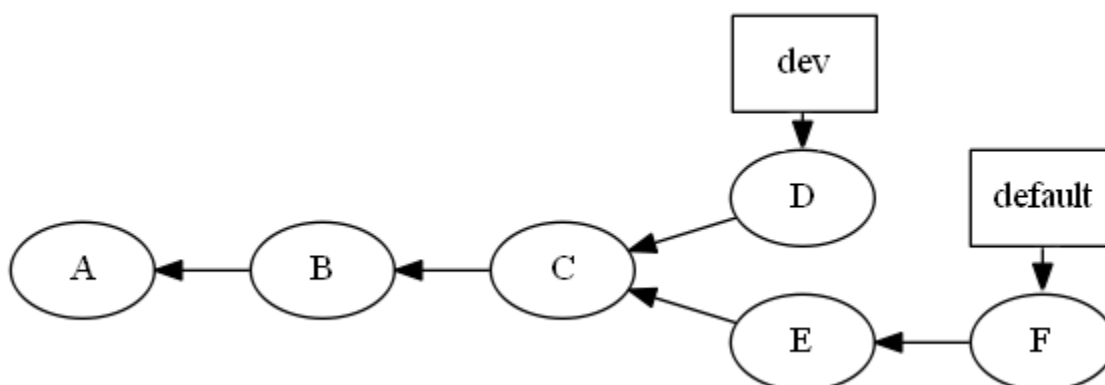


Рисунок 18. Пример репозитория после клонирования части веток

Обратите внимание, что при выполнении команд `push` и `pull` Mercurial по умолчанию выполняет действия со всеми ветками. Если вы

ХОТИТЕ ВЫТЯНУТЬ ИЛИ ВЫТОЛКНУТЬ ИЗМЕНЕНИЯ ТОЛЬКО В ОДНОЙ ВЕТКЕ, ВЫ ДОЛЖНЫ ИСПОЛЬЗОВАТЬ ОПЦИЮ `-b` (см. рис. 19).

```
$ hg pull -b default
pulling from https://gvterechov@bitbucket.org/gvterechov/preg
searching for changes
adding changesets
adding manifests
adding file changes
added 136 changesets with 266 changes to 100 files
$ hg push -b default
pushing to https://gvterechov@bitbucket.org/gvterechov/preg
searching for changes
no changes found
```

Рисунок 19. Пример вытягивания и выталкивания в одной ветке в Mercurial

1.4 "Легкие" ветки в Mercurial

В Mercurial существует еще один способ ветвления репозитория - "закладки", bookmarks. Работа закладок в Mercurial похожа ветки в git: они не хранятся постоянно, а существуют лишь как именованные указатели на вершины веток, поэтому могут быть полностью удалены. Поэтому закладки рекомендуется использовать при необходимости создавать значительное количество временных веток (стратегии `issue-branch` и иногда `feature-branch`).

Закладка - это ссылка на коммит, которая автоматически обновляется при добавлении новых коммитов поверх него. Команды `bookmark` и `bookmarks` работают аналогично командам `branch` и `branches`. Если вы выполните команду `hg bookmark foo`, созданная закладка `foo` будет ссылаться на текущий набор изменений (см. рис. 20).

```
$ hg bookmark foo
```

Рисунок 20. Пример создания закладки в Mercurial

По мере добавления коммитов активная закладка будет перемещаться вперед с каждым коммитом. Закладки всегда указывают на последнюю ревизию в направлении ("ветке") работы. Вы можете переключаться между закладками так же, как и между ветками, используя команду `update`, использовать имя закладки в качестве аргумента команды `merge` и т.д (см. рис. 21).

```
$ hg bookmark bar
$ hg update foo
0 files updated, 0 files merged, 0 files removed, 0 files
unresolved
$ hg update bar
0 files updated, 0 files merged, 0 files removed, 0 files
unresolved
```

Рисунок 21. Пример переключения между закладками в Mercurial

Закладка может быть удалена с помощью команды `hg bookmark - delete` с указанием имени закладки (см. рис. 22). Однако, в отличие от Git, удаление не слитой закладки удаляет только саму закладку (имя), оставляя коммиты этой ветки на месте (ветка становится анонимной). Вы можете закрыть ветку предварительно, используя `commit --close-branch`.

```
$ hg bookmark --delete bar
```

Рисунок 22. Пример удаления закладки в Mercurial

В современных версиях Mercurial приняты следующие правила работы с закладками на удаленных серверах. При клонировании или вытягивании из репозитория передаются все закладки. При выталкивании же выталкиваются все закладки, которые есть как в локальном, так и в удаленном репозиториях; если вы хотите вытолкнуть на сервер новую для него закладку это надо указать команде `push` специальной опцией `-B <имя закладки>`. Таким образом вы можете держать часть закладок чисто для локальной работы, не делаясь ими с окружающими. Этот прием хорошо согласуется с возможностью полностью удалить закладку позднее.

1.5 Работа с метками в Mercurial и Git

Метки (tags) – это возможность давать определенным (важным) ревизиям проекта человеко-читаемые имена. В отличие от веток, указатели которых обновляются после нового коммита, метки продолжают указывать на ту же ревизию. Например, при использовании стратегии `major release-branch` на каждый большой релиз выделяется отдельная ветка, в рамках которой делается потом несколько малых релизов. Для обеспечения возможности легко получить код малого релиза командой `update` на этот коммит ставится метка (см. рис. 23).

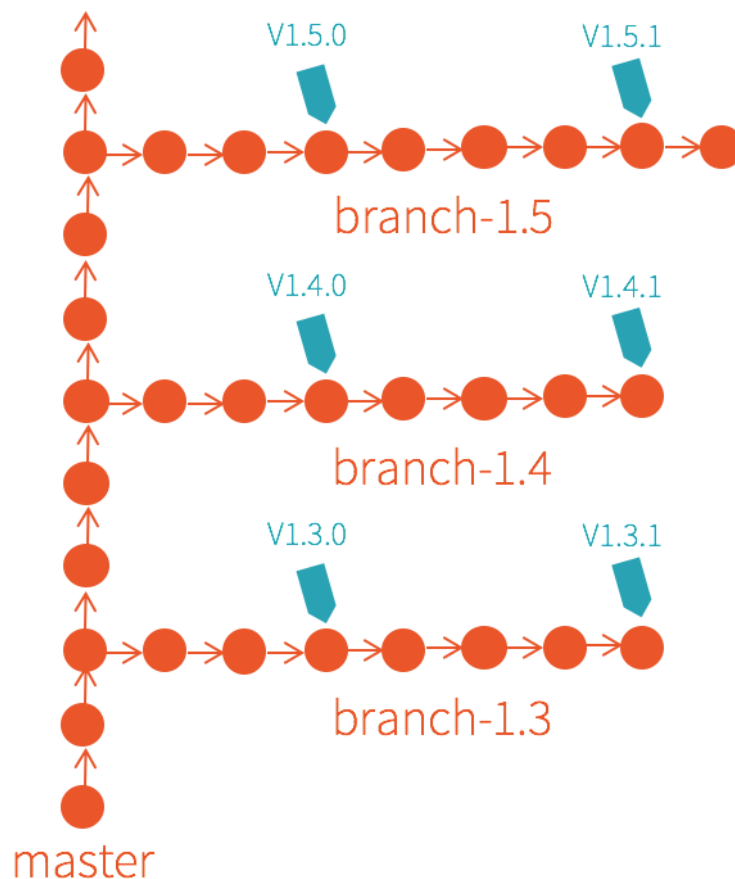


Рисунок 23. Пример репозитория с ответвлениями на большие релизы и расставленными метками на малые релизы

В Mercurial метки хранятся в специальном версионизируемом файле с именем `.hgtags`. Это сделано для того, чтобы сами метки (и процесс их изменения) были версионизируемыми, их можно было сливать при слиянии веток, добавлять теги к уже существующим коммитам и изменять положение тегов. Однако это создает некоторые странные ситуации, к которым необходимо быть готовым. Так метка какого-либо коммита не может быть частью того коммита, на который она указывает – она должна быть добавлена после, причем ее добавление необходимо закоммиттить. При слиянии файла с метками возможно возникновение конфликтов слияния, причем Mercurial не занимается их специальным разрешением – это должен делать пользователь – обычно конфликт возникает при добавлении меток в обеих ветках и при его разрешении необходимо взять обе добавленные строки. Наконец, если вы обновитесь до ревизии с какой-либо меткой, вы не обнаружите этой метки (ибо она была добавлена более поздней ревизией). Команды типа `clone`, `push` и `pull` работают со всеми метками автоматически (т.к. они являются частью файловой истории), а команда `merge` берет на себя слияние меток из разных репозиториях.

Для создания ветки используется команда `hg tag <имя метки>`, которая создает метку, указывающую на предыдущий коммит (от которого непосредственно происходит содержимое рабочей копии). Для ее удаления используется эта же команда с ключом `--remove`. Команда `update` позволяет переключиться к отмеченной ревизии, а команда `tags` показывает список всех меток, доступных на данный момент (см. рис. 24).

```
$ hg tag dev_tag
$ hg tags
tip                                     6070:5210c7425f67
длина названия теста увеличена до 255 5625:5020f2fc1aea
добавлен код для создания message_provider'a 5547:760b293defe3
добавлен промежуточный этап для импорта 5535:ef2929d4e490
Удаление заданий - интерфейс 5485:e050146dbaac
dev_tag                                4612:d29aee1e939d
$ hg update tip
2 files updated, 0 files merged, 0 files removed, 0 files
unresolved
$ hg tag --remove dev_tag
```

Рисунок 24. Пример работы с метками в Mercurial

Если вы хотите завести метку для себя работая с Mercurial, то можете использовать ключ `-l`, создаваемые с его помощью метки будут принадлежать только вашему репозиторию.

В Git метки представляют из себя указатели на коммиты и делятся на две категории: легковесные (по сути ветка, указатель на которую навсегда заморожен на определенном коммите) и аннотированные. Аннотированные ветки являются объектом подобно наборам изменений – они содержат помимо имени запись об авторе, могут содержать сообщение и могут быть подписанными. Создаются метки командой `tag`, при этом для создания аннотированной метки применяется опция `-a`, для указания сообщения `-m`, а если вы хотите подписать метку с помощью GPG – вы используете опцию `-s` вместо `-a`. Для создания легковесной метки не должно указываться опций `-s`, `-a` и `-m`. Просмотреть метки можно с помощью команды `tag` без параметров (см. рис. 25).

```
$ git tag
v0.1
v1.3
```

Рисунок 25. Пример просмотра меток в Git

Git по умолчанию не выталкивает метки. Вы можете вытолкнуть конкретную метку, указав ее имя в опциях команды `push`, или использовать опцию `--tags` для отправки на удаленный сервер всех меток. При вытягивании или клонировании удаленного сервера, однако, получают все находящиеся там метки.